Outline

- Basic Concepts
- Frequent Item Set Mining Methods
  - Apriori Algorithm
  - FP-growth
- Strengths and Weaknesses

What Is Frequent Pattern Analysis?

- Frequent pattern: A pattern (a set of items, subsequences, substructures, etc.) that occurs frequently in a data set. E.g, a set of items, such as milk and bread, that appear frequently together in a transaction data set is a frequent item set.

- A subsequence, such as buying first a PC, then a digital camera, and then a memory card, if it occurs frequently in a shopping history database, is a (*frequent)* sequential pattern.

- Finding frequent patterns plays an essential role in mining associations, correlations, and many other interesting relationships among data.

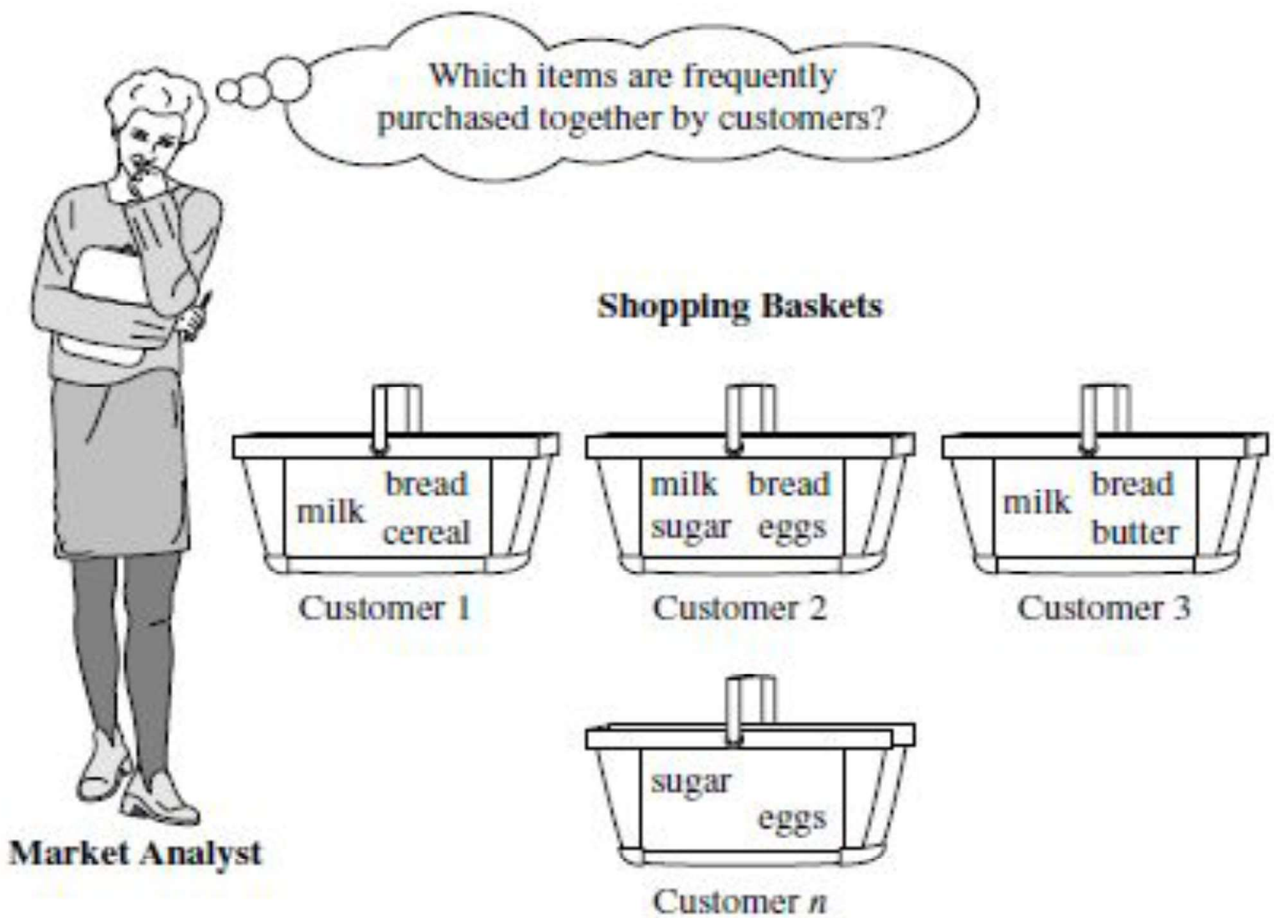
What Is Frequent Pattern Analysis?

- Motivation: Finding inherent regularities in data
  - What products were often purchased together?— Beer and diapers?!
  - What are the subsequent purchases after buying a PC?
  - What kinds of DNA are sensitive to this new drug?
  - Can we automatically classify web documents?

What Is Frequent Pattern Analysis?

- Applications
  - Basket data analysis, catalog design, sale campaign analysis, Web log (click stream) analysis, and DNA sequence analysis.
- Market basket analysis
- This process analyzes customer buying habits by finding associations between the different items that customers place in their “shopping baskets” (Figure).
- The discovery of these associations can help retailers develop marketing strategies by gaining insight into which items are frequently purchased together by customers.
- E.g. if customers are buying milk, how likely are they also buy bread (and what kind of bread) on the same trip to the supermarket.

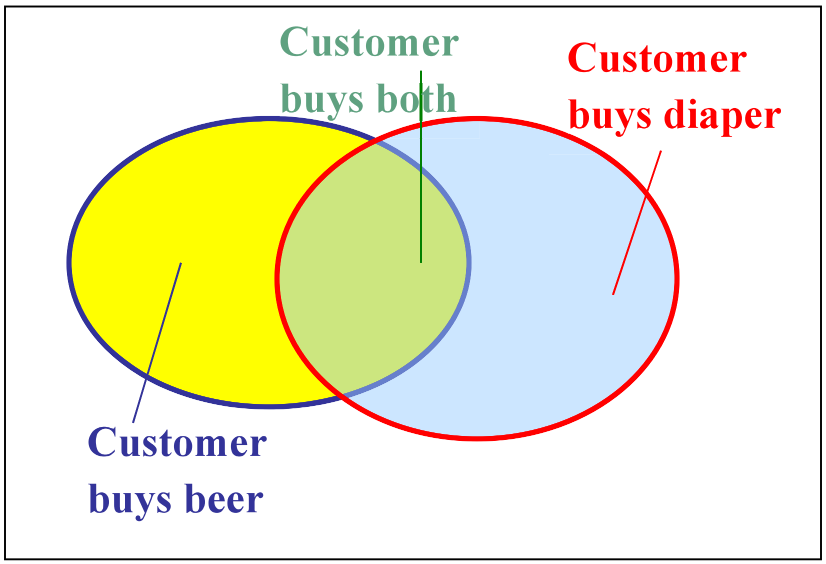
Market Basket Analysis



Market basket analysis.

Basic Concepts: Frequent Patterns

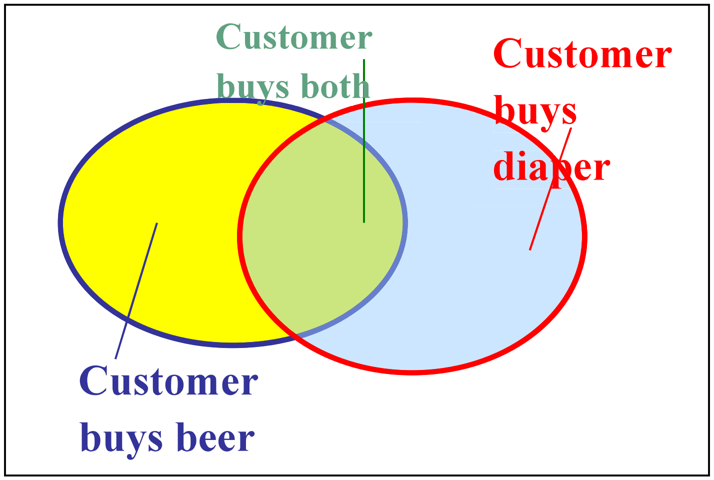
Tid	Items bought
10	Beer, Nuts, Diaper
20	Beer, Coffee, Diaper
30	Beer, Diaper, Eggs
40	Nuts, Eggs, Milk
50	Nuts, Coffee, Diaper, Eggs, Milk



- Itemset: A set of one or more items
- k-itemset $X = \{x_1, \ldots, x_k\}$
- (absolute) support, or, support count of X: Frequency or occurrence of an itemset X
- (relative) support, s, is the fraction of transactions that contains X (i.e., the probability that a transaction contains X)
- An itemset X is frequent if X's support is no less than a min-sup threshold

Basic Concepts: Association Rules

Tid	Items bought
10	Beer, Nuts, Diaper
20	Beer, Coffee, Diaper
30	Beer, Diaper, Eggs
40	Nuts, Eggs, Milk
50	Nuts, Coffee, Diaper, Eggs, Milk

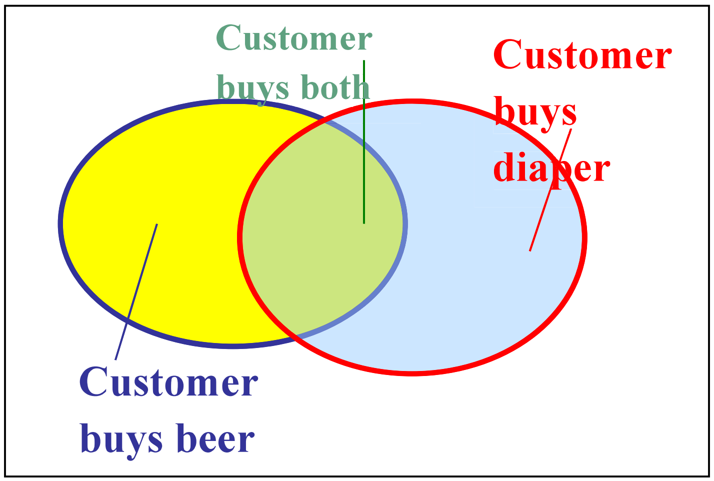


- Find all the rules *A* □ *B* with minimum support and confidence
  - support, *s*, probability that a transaction contains A ∪ B
  - confidence, *c*, conditional probability that a transaction having A also contains B
  - support (A □ B) = P (A U B)
  - Confidence (A □ B) = P(B|A)
  - P(B|A) =

    Support-count (A U B) / support-count (A)

Basic Concepts: Association Rules

Tid	Items bought
10	Beer, Nuts, Diaper
20	Beer, Coffee, Diaper
30	Beer, Diaper, Eggs
40	Nuts, Eggs, Milk
50	Nuts, Coffee, Diaper, Eggs, Milk



Let minsup = 50%, minconf = 50%

Freq. Pattern: Beer:3, Nuts:3, Diaper:4, Eggs:3, {Beer, Diaper}:3

- Association rules: (many more!)
  Beer □ *Diaper* (3/5*100 = 60%, 3/3*100 = 100%)
  Diaper □ *Beer* (3/5*100 = 60%, 3/4*100 = 75%)

- A support of 60% means that 60% of all the transactions show that beer and diaper are purchased together.

- A confidence of 100% means that 100% of the customers who purchased a beer also bought the diaper.

Rules that satisfy both a *min- sup* and a *min-conf* are called **strong**.

Outline

- Basic Concepts
- Frequent Item Set Mining Methods
  - Apriori: A Candidate Generation-and-Test Approach
  - FP-growth
- Strengths and Weaknesses

The Downward Closure Property and Scalable Mining Methods

- The downward closure property of frequent patterns
  - Any non-empty subset of a frequent item set must be frequent.
  - If **{beer, diaper, nuts}** is frequent, so is **{beer, diaper}.**
    - i.e., every transaction having {beer, diaper, nuts} also contains {beer, diaper}.
- Scalable mining methods: Three major approaches
  - Apriori (Agrawal & Srikant@VLDB'94).
  - Freq. pattern growth (FPgrowth—Han, Pei & Yin @SIGMOD'00).
  - Vertical data format approach (Charm—Zaki & Hsiao @SDM'02).

Apriori: A Candidate Generation & Test Approach

- Apriori pruning principle: If there is any itemset which is infrequent, its superset should not be generated/tested!
- Method:
  - Initially, scan DB once to get frequent 1-itemset
  - Generate length (k+1) candidate itemsets from length k frequent itemsets.
  - Test the candidates against DB.
  - Terminate when no frequent or candidate set can be generated.

Implementation of Apriori

- How to generate candidates?
  - Step 1: self-joining L_k
  - Step 2: pruning
- Example of Candidate-generation
  - $L_3=\{abc, abd, acd, ace, bcd\}$
  - Self-joining: L_3*L_3
    - *abcd* from *abc* and *abd*
    - *acde* from *acd* and *ace*
  - Pruning:
    - *acde* is removed because *ade* is not in L_3
  - $C_4 = \{abcd\}$

The Apriori Algorithm—An Example

$Sup_{min} = 2$

Database TDB

Tid	Items
10	A, C, D
20	B, C, E
30	A, B, C, E
40	B, E

1st scan

C_1

Itemset	sup
{A}	2
{B}	3
{C}	3
{D}	1
{E}	3

L_1

Itemset	sup
{A}	2
{B}	3
{C}	3
{E}	3

C_2

Itemset
{A, B}
{A, C}
{A, E}
{B, C}
{B, E}
{C, E}

2nd scan

C_2

Itemset	sup
{A, B}	1
{A, C}	2
{A, E}	1
{B, C}	2
{B, E}	3
{C, E}	2

L_2

Itemset	sup
{A, C}	2
{B, C}	2
{B, E}	3
{C, E}	2

C_3

Itemset
{B, C, E}

3rd scan

L_3

Itemset	sup
{B, C, E}	2

Generating Association Rules

- The data contain frequent item set X = {B, C, E}. What are the association rules that can be generated from X? The nonempty subsets of X are {B, C}, {B, E}, {C, E}, {B}, {C}, {E}. The resulting association rules are:
  - {B, C} □ {E} , confidence = 2/2 = 100%
  - {B, E} □ {C} , confidence = 2/3 = 66%
  - {C, E} □ {B} , confidence = ?
  - {B} □ {C, E} , confidence = ?
  - {C} □ {B, E} , confidence = ?

Tid	Items
10	A, C, D
20	B, C, E
30	A, B, C, E
40	B, E

- If the minimum confidence threshold is, say, 70%, then Which ones are selected?,

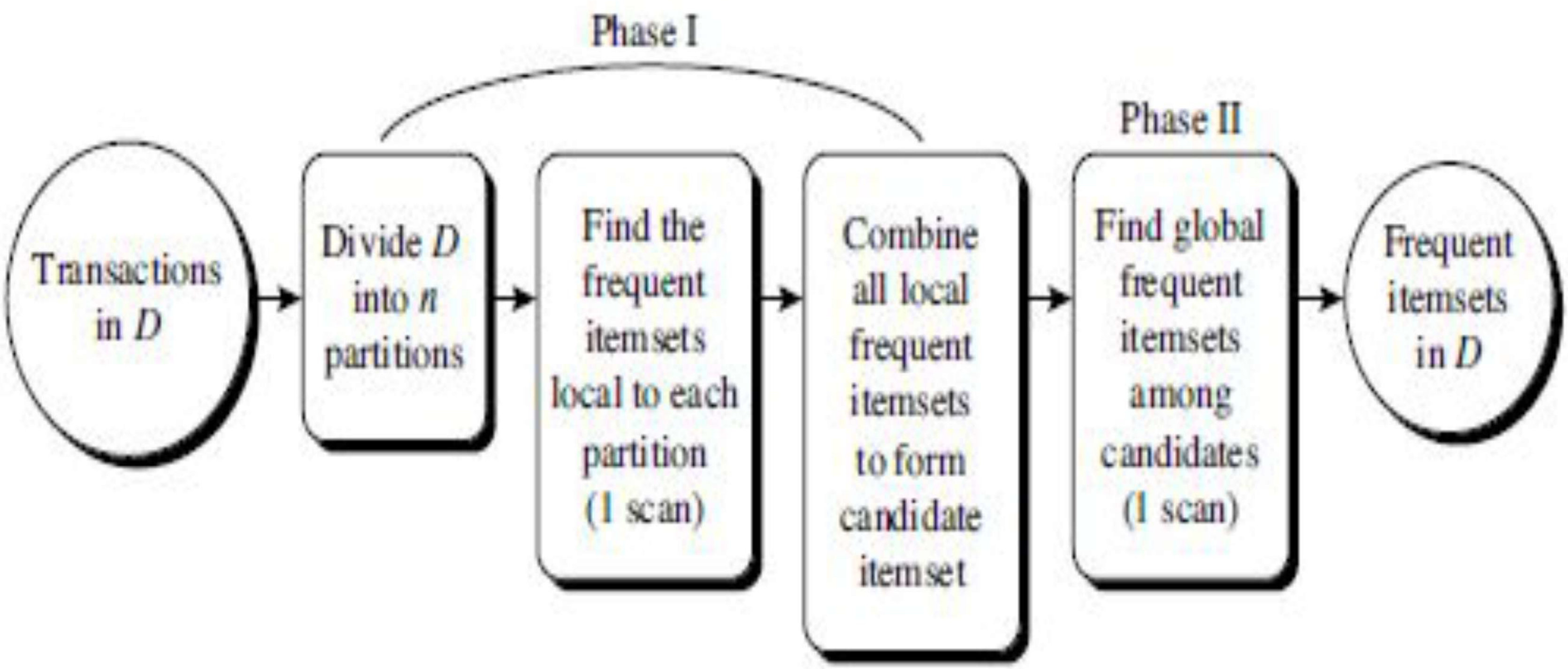
Outline

- Basic Concepts
- Frequent Item Set Mining Methods
  - Apriori: A Candidate Generation-and-Test Approach
    - Improving the efficiency of Apriori
  - FP-growth
- Strengths and Weaknesses

Further Improvement of the Apriori Method

- Major computational challenges
  - Multiple scans of transaction database
  - Huge number of candidates
  - Tedious workload of support counting for candidates
- Improving Apriori: general ideas
  - Reduce passes of transaction database scans
  - Shrink number of candidates
  - Facilitate support counting of candidates

Partition: Scan Database Only Twice



Partition: Scan Database Only Twice

- Requires two database scans to mine the frequent items.
- It consists of two phases: In phase I, the algorithm divides the transactions of D into n non overlapping partitions.
- If the minimum relative support threshold is min_sup, then the minimum support count for a partition is min_sup * the number of transactions in that partition.
- For each partition, all the local frequent item sets are found.
- A local frequent itemset may or may not be frequent with respect to the entire database.

Partition: Scan Database Only Twice

- However, any itemset that is potentially frequent in D must occur as a frequent itemset in at least one of the partitions.
- Therefore, all local frequent item sets are candidate item sets with respect to D. The collection of frequent item sets from all partitions form the global candidate item sets concerning D.
- In phase II, the second scan of D is conducted, in which the actual support of each candidate is assessed to determine the global frequent item sets.

Any Question